AI Usage Log — Whole-Assessment Record

QA Take-Home Assessment | AI Usage Disclosure & Transparency Log

Author: Uma Uka **Primary Tool:** Claude Code (Anthropic) **Date:** August 31, 2026

Scope & Overview

Scope of this log: Claude Code (Anthropic) was used as an interactive pair-programming assistant for Part 2 (API test automation) and Part 5 (CI/CD), across a single continuous session. This log records that session's real prompts, actions, and corrections.

Part 1 (test plan/test cases) and Part 6 Task A (AI critique) involved AI assistance in a separate tool/session. Usage is documented here to maintain complete transparency across all assessment parts.

1. What AI Was Used For

- **Architecture Scaffolding:** Scaffolding the Playwright API test suite architecture for Part 2 (`fixtures/helpers/config/tests` split) from a detailed brief, with an explicit planning step and sign-off before writing code.
- **Iterative Spec Development:** Iteratively building each spec file (auth, CRUD, negative/boundary, filtering) one at a time, verifying real behavior against the running local Restful Booker instance via direct HTTP requests before writing assertions.
- **CI/CD Workflow Engineering:** Designing and building the GitHub Actions CI/CD workflow for Part 5: job parallelization, Restful Booker service container, test sharding, artifact upload, and the `test.fail()` known-bug CI signal, followed by verifying actual execution on GitHub Actions.
- **Document Generation:** Generating a formatted PDF bug report (`docs/Restful_Booker_Bug_Report.pdf`) via Python script automation.
- **Documentation Maintenance:** Drafting and maintaining `README.md`, including catching and fixing stale file-path references after document renames.
- **Git Workflow Management:** Supporting git workflow: staging, committing, and pushing in small increments after each reviewed unit of work.

2. Example Prompts & Actions Taken

Prompt 1 — Initial Scope-Setting & Planning Gate

```
"I'm building the API test automation suite (Part 2) and CI/CD pipeline (Part 5) for a QA
engineer take-home assessment. Read this whole brief, then STOP and show me your planned file
structure and test list before writing any code — I need to sign off on the approach first..."
```

Action Taken: Produced a full plan (file structure, test list per spec file, dependency choices) and entered a review gate before writing any code, rather than generating the suite in one unreviewed pass.

Prompt 2 — Correcting Tech Stack & Workflow Cadence

```
"I'm using JavaScript not TypeScript. I want to commit each spec/test grouping to the repo so I will need you to pause and commit each after each. eg. when we do auth part we commit and push to remote repo."
```

Action Taken: Revised the plan from TypeScript to plain JavaScript, and switched to committing and pushing after each spec file was built and verified, rather than doing one final commit dump at the end.

Prompt 3 — Requesting Formatted Deliverable

```
"Yes, draft the Bug-Report content with severities and put it in a well formatted pdf file"
```

Action Taken: Assigned severities to each defect with stated reasoning, wrote and executed a Python generation script to produce a formatted PDF, and visually inspected rendered pages before presenting.

Prompt 4 — Formatting & Layout Correction

```
"Could you change § to section and give a line space between Bug Report heading and the rest of the document"
```

Action Taken: Located the single '§' symbol occurrence and title-spacing issue in the PDF generation script, fixed both, regenerated the PDF, and re-verified layout visually before confirming.

3. What Was Corrected in AI Outputs

- **Tech Stack Assumption:** The initial AI plan defaulted to TypeScript based on general mentions in the brief; the author corrected this to plain JavaScript before any code was written.
- **Delivery & Commit Cadence:** The default AI approach would have built the entire suite before committing; the author enforced committing and pushing after each reviewed spec file instead.
- **Self-Caught Testing Reporter Gap:** Local manual test runs used an `--reporter=list` override for cleaner terminal output, which silently skipped JSON/JUnit reporters defined in config. This was caught before trusting CI behavior by re-running without overrides.
- **Stale Document References:** The README referenced `docs/Bug-Report.pdf` after the file was replaced with `Restful_Booker_Bug_Report.pdf`. Caught during accuracy review and updated.
- **PDF Formatting Nits:** Corrected raw section symbols ('§') and heading spacing upon explicit request.

4. Validation Approach

Every claim about Restful Booker's actual behavior — auth response shape, status codes on protected endpoints, delete semantics, filter behavior — was verified against the running local instance via direct HTTP requests or actual Playwright runs before being encoded into test assertions or written into the bug report, rather than assumed from general knowledge of "typical" REST APIs.

Similarly, the CI/CD pipeline's behavior was confirmed against a real, completed GitHub Actions run (checked job-by-job via the GitHub API) rather than treated as correct solely on the basis of a committed YAML workflow file.